Getting Started with Cursor – Fundamentals

Understanding the Core Concepts

What You'll Learn

- What is Cursor?
- How the Agent works
- Modes: Agent, Plan, Ask
- Models & Pricing deep dive
- Your first workflow

Speaker Notes: "This section covers everything you need to know to start using Cursor effectively. We'll focus on understanding the core concepts before diving into code."

What Is Cursor? (Review)

- **AI-first code editor** built on VS Code
- **Autonomous agent** that can complete complex coding tasks
- Reads your entire codebase to understand context
- Edits files, runs terminal commands, and creates PRs
- Works with multiple AI models (OpenAI, Anthropic, Google, Cursor)

Speaker Notes: "Cursor isn't just ChatGPT in an editor. It's a fully autonomous developer that works alongside you."

The Agent – Your AI Developer

```
User Prompt → Agent → Tools → Result
                 ↓
          • Reads files
          • Searches code
          • Edits files
          • Runs commands
          • Browses web
```

Key Characteristics

- The **Agent** is the core AI assistant
- It has access to **tools** (file ops, terminal, search, browser)
- Can work **autonomously** for multi-step tasks
- Maintains **conversation context** across messages

Speaker Notes: "Think of the Agent as a junior developer you can delegate tasks to. It

Agent Components (3 Pillars)

Component	What it is	Who controls
Instructions	Rules and system prompts that guide behavior	You + Cursor defaults
Tools	Capabilities (read, write, search, terminal, browser)	Built into Cursor
Model	The AI brain (Claude, GPT, Composer, etc.)	You choose

Speaker Notes: "The Agent's behavior is the combination of these three things. You can customize instructions with Rules, choose different models for different tasks, and the tools are always available."

The Three Modes

Mode	Icon	What it does	When to use
Agent	🤖	Full access to all tools – reads, writes, runs commands	Most tasks – building features, fixing bugs
Plan	📋	Designs first, asks clarifying questions, creates plan, then builds	Complex features, unclear requirements
Ask	🔍	Read-only – searches and answers, never edits files	Exploring code, learning the codebase

Shortcut: `Shift+Tab` rotates between modes

Speaker Notes: "Agent mode is your default. Plan mode is for when you want to think through a problem before coding. Ask mode is for when you just want answers without any changes."

Mode Comparison – Deep Dive

Capability	Agent	Plan	Ask
Read files	✅	✅	✅
Search codebase	✅	✅	✅
Edit files	✅	❌ (creates plan only)	❌
Run terminal commands	✅	❌	❌
Ask clarifying questions	❌	✅	❌
Create implementation plan	❌	✅	❌
Requires approval before edit	❌	✅ (after plan)	N/A

Speaker Notes: "Notice the trade-offs. Agent mode is most powerful but can make changes immediately. Plan mode is safer for complex work. Ask mode is completely safe for exploration."

How to Switch Modes

Methods

Method	Action
Keyboard	`Shift+Tab` (rotate through modes)
Slash command	`/agent`, `/plan`, `/ask`
CLI flag	`agent --mode plan`, `agent --plan`

Screenshot: *Mode picker dropdown in Cursor*

Speaker Notes: "The fastest way is `Shift+Tab`. You'll see the mode change in the chat input. The slash commands work in the CLI too."

Understanding Models – Why It Matters

- Different models have different **strengths** (speed, cost, intelligence)
- Choosing the right model saves **money** and **time**
- Model choice affects:
  - **Quality** of code generated
  - **Speed** of response
  - **Cost** per task

Speaker Notes: "You wouldn't use a sledgehammer to hang a picture. Same with models – use cheap/fast models for simple tasks, expensive/smart models for complex problems."

Models – Complete Comparison Table (1/2)

Model	Provider	Speed	Intelligence	Cost	Best for
GPT-5 Mini	OpenAI	Medium	Low-Medium	$	Simple fixes, typos
Gemini 2.5 Flash	Google	Fast	Medium	$	Quick answers
Gemini 3 Flash	Google	Fast	Medium	$	Fast responses
Composer 2 (Std)	Cursor	Fast	Frontier	$$	**Everyday coding – best value**
Claude 4.5 Haiku	Anthropic	Fast	High	$$	Balanced speed/quality
GPT-5	OpenAI	Medium	High	$$	General coding
Grok 4.3	xAI	**Fastest**	High	$$	Speed-critical tasks
GPT-5.3 Codex	OpenAI	Medium	Frontier	$$	**Coding specialized – great value**

Speaker Notes: "The 'best value' sweet spot is Composer 2 and GPT-5.3 Codex. They deliver frontier-level quality at mid-tier prices."

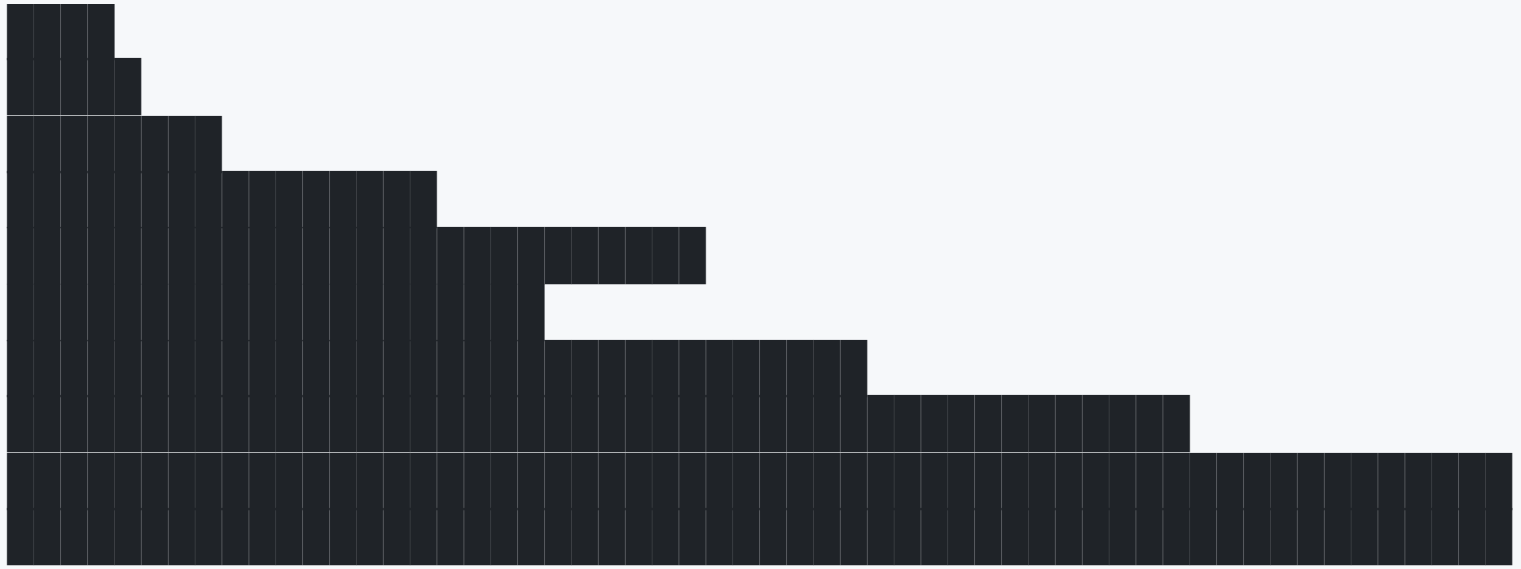
Models – Complete Comparison Table (2/2)

Model	Provider	Speed	Intelligence	Cost	Best for
Gemini 3.1 Pro	Google	Medium	Frontier	$$$	**Image processing + frontend**
Claude 4.7 Opus	Anthropic	Medium	Frontier	$$$$	Maximum quality, architecture
GPT-5.5	OpenAI	Medium	Frontier	$$$$	Persistence, long sessions
Claude 4.6 Opus (Fast)	Anthropic	**Very Fast**	High	$$$$$	Emergency speed

Speaker Notes: "The frontier models (Opus, GPT-5.5) are the smartest, but they cost significantly more. Use them only for the hardest 10% of your tasks."

Model Pricing Comparison (Input Cost)

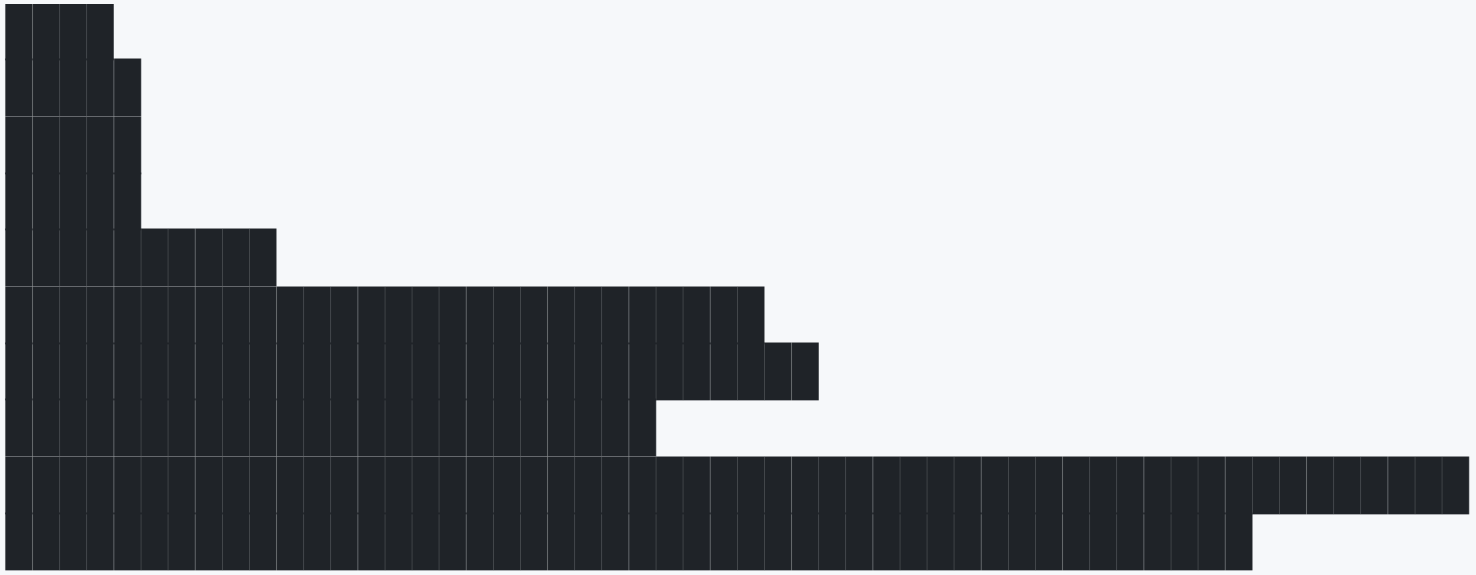
```
GPT-5 Mini         $0.25
Gemini 2.5 Flash   $0.30
Composer 2         $0.50
Claude 4.5 Haiku   $1.00
GPT-5.3 Codex      $1.75
Grok 4.3           $1.25
Gemini 3.1 Pro     $2.00
Claude 4.6 Sonnet  $3.00
GPT-5.5            $5.00
Claude 4.7 Opus    $5.00
```



Speaker Notes: "Notice the range – from $0.25 to $5.00 per million input tokens. That's a 20x difference."

Model Pricing Comparison (Output Cost)

```
GPT-5 Mini         $2.00
Composer 2         $2.50
Grok 4.3           $2.50
Gemini 2.5 Flash   $2.50
Claude 4.5 Haiku   $5.00
GPT-5.3 Codex      $14.00
Claude 4.6 Sonnet  $15.00
Gemini 3.1 Pro     $12.00
GPT-5.5            $30.00
Claude 4.7 Opus    $25.00
```



Speaker Notes: "Output cost is where the difference really shows. Output tokens are typically 5-10x more expensive than input."

Model Selection Decision Tree

```
START: What are you doing?

Simple fix (typo, text change)?
├─ YES → GPT-5 Mini (cheapest)
└─ NO → Continue

Need image processing (UI, screenshots)?
├─ YES → Gemini 3.1 Pro (only one!)
└─ NO → Continue

Everyday coding, debugging, features?
├─ YES → Composer 2 or GPT-5.3 Codex (best value)
└─ NO → Continue

Complex multi-step, long session?
├─ YES → GPT-5.5 or Claude Opus
```

Speaker Notes: "For 80% of tasks, Composer 2 is the answer."

Understanding Tokens & Context

```
[=== 200K tokens ===] Default context
[======== 1M tokens ========] Max Mode

1 token ≈ 0.75 words
200K tokens ≈ 150 pages of text
1M tokens ≈ 750 pages of text
```

Term	Meaning
Token	Smallest unit of text AI processes (~0.75 words)
Context window	How much text AI can "see" at once
Max Mode	Extends context to maximum (costs more tokens)

Speaker Notes: "Context is the AI's working memory. Larger context = AI can see more of your codebase at once, but costs more tokens."

Pricing Plans – Individual

Plan	Monthly Price	Included API Usage	Best for
Pro	$20	$20	Occasional users (1-2 hours/day)
Pro Plus	$60	$70	Daily users (3-5 hours/day)
Ultra	$200	$400	Power users (all day)

What's included in ALL plans

- ✅ Unlimited tab completions
- ✅ Extended agent usage limits
- ✅ Access to Bugbot
- ✅ Access to Cloud Agents

Speaker Notes: "Start with Pro. You can always upgrade mid-month. Most individuals never exceed the $20 included usage."

What Happens When You Hit Your Limit?

```
You use included $20
        ↓
Optional: Upgrade plan (higher tier)
        ↓
Or: Continue with on-demand (pay-as-you-go)
        ↓
Service never degrades – you just pay a little extra
```

Speaker Notes: "You never get cut off. Your AI doesn't slow down. You just pay a small overage at the end of the month. You can also set spending limits in settings."

Real-World Cost Examples

Task	Tokens	Best Model	Estimated Cost
Fix a typo	1K in / 200 out	GPT-5 Mini	$0.00065
Add a console log	2K in / 500 out	GPT-5 Mini	$0.0015
Explain a function	5K in / 1K out	Composer 2	$0.005
Small feature (UI)	20K in / 5K out	Composer 2	$0.025
Debug an error	50K in / 10K out	GPT-5.3 Codex	$0.23
Refactor a file	100K in / 30K out	GPT-5.3 Codex	$0.60
New feature (full)	200K in / 100K out	Claude Opus	$3.50

Speaker Notes: "Most tasks cost under 10 cents. You'd need to do hundreds of tasks to hit $20."

Teams Pricing

Plan	Price	Included usage per user
Teams	$40/user/month	$20
Enterprise	Custom	Pooled usage

Additional

- **Cursor Token Rate:** $0.25 per million tokens (all non-Auto requests)
- **On-demand usage:** Automatic after included amount
- **Prorated billing:** Pay only for days user is active

Speaker Notes: "Teams billing is per active user, not pre-allocated seats."

Cursor Token Rate Explained

- Applies to **all non-Auto agent requests**
- Covers:
  - Semantic search
  - Custom model execution (Tab, Apply, etc.)
  - Infrastructure and processing costs
- Applied to **input, output, and cached tokens**
- Also applies to **BYOK** (bring your own key)

Example: 1M tokens of Claude Sonnet ($3 + $15 = $18), add $0.25 = **$18.25 total**

Speaker Notes: "The Token Rate is small (0.25 cents per million tokens) but adds up. It covers Cursor's infrastructure costs."

Your First Workflow – Step by Step

Step	Action
1	Open Cursor (already installed)
2	Open a project folder
3	Press `Ctrl+I` to open Agent
4	Type: "*Explain this codebase to me*"
5	Read the explanation
6	Type: "*Suggest 3 small improvements*"
7	Pick one improvement
8	Type: "*Make improvement #1*"

Speaker Notes: "This is your 10-minute onboarding. Do this once and you'll understand 90% of what you need to know."

Keyboard Shortcuts – Fundamentals

Action	Windows/Linux	Mac
Open Agent	`Ctrl+I`	`Cmd+I`
Rotate modes	`Shift+Tab`	`Shift+Tab`
Queue message	`Enter`	`Enter`
Send immediately	`Ctrl+Enter`	`Cmd+Enter`
New chat	`Ctrl+Shift+N`	`Cmd+Shift+N`

Speaker Notes: "Learn these five shortcuts. They'll save you hours over time."

Common Pitfalls & How to Avoid

Pitfall	Fix
Vague prompts	Be specific: "Change the login button color to #0000FF"
No context	Use @mentions to point at relevant files
Using Opus for simple tasks	Start with Composer 2, upgrade only if needed
Ignoring checkpoints	Click any checkpoint to undo mistakes
Not reviewing diffs	Always review before trusting changes

Speaker Notes: "Most issues come from vague prompts. Give the agent exact file names and expected outcomes."

Summary – Key Takeaways

1. **Agent mode** is your default – Plan mode for complex work, Ask for exploration
2. **Composer 2** and **GPT-5.3 Codex** are the best value models for most tasks
3. **Most tasks cost under 10 cents** – you get $20/month included
4. **Use @mentions** when you know which files matter
5. **Checkpoints** are your undo button – use them
6. `Shift+Tab` rotates modes – fastest way to switch

Speaker Notes: "You now know enough to be productive. The rest is practice and learning the specific tools."

Q&A / Practice Challenge

Challenge

1. Open any code project
2. Ask Agent: "*Explain the main entry point of this codebase*"
3. Ask: "*Suggest 2 small improvements*"
4. Have Agent make one change
5. Review the diff
6. Verify with tests

Resources

- Documentation: `cursor.com/docs`
- CLI help: `agent --help`

Appendix – Model ID Reference

Model	ID to use in CLI/API
Composer 2	`composer-2`
GPT-5 Mini	`gpt-5-mini`
GPT-5.3 Codex	`gpt-5.3-codex`
Claude 4.6 Sonnet	`claude-4.6-sonnet`
Claude 4.7 Opus	`claude-4.7-opus`
Gemini 3.1 Pro	`gemini-3.1-pro`
Grok 4.3	`grok-4.3`
GPT-5.5	`gpt-5.5`

Speaker Notes: "Use these IDs in CLI commands like `agent --model composer-2` or in API calls."

Appendix – Cost Per 1M Tokens (Full Table)

Model	Input	Output	Cache Read	Cache Write
GPT-5 Mini	$0.25	$2.00	$0.025	-
Composer 2	$0.50	$2.50	$0.20	-
GPT-5	$1.25	$10.00	$0.125	-
Grok 4.3	$1.25	$2.50	$0.20	-
Claude 4.5 Haiku	$1.00	$5.00	$0.10	$1.25
GPT-5.3 Codex	$1.75	$14.00	$0.175	-
Gemini 3.1 Pro	$2.00	$12.00	$0.20	-
Claude 4.6 Sonnet	$3.00	$15.00	$0.30	$3.75

Speaker Notes: "Keep this table handy. When in doubt, start with Composer 2 and adjust based on results."

Thank You!

Questions?

Cursor Documentation: cursor.com/docs

Support: cursor.com/help

Community: cursor.com/discord